

Test # 8

You've survived the test. The results will arrive soon. 🎉

Your score: 82%18.75 / 30

Duration: 0:23:18

1. Consider:

```
class Employee implements Comparable<Employee> {  
    Long id;  
    String name;  
    @Override  
    public int compareTo(Employee e) {  
        return this.name.compareTo(e.name);  
    }  
}
```

1 / 1 point

Then,

```
Set<Employee> employees = new TreeSet<>();  
employees.add(new Employee(101L, "Rahul");  
employees.add(new Employee(102L, "Rahul");
```

What is the most important issue?

Your Answer: ☒ Correct

☐ TreeSet will throw ClassCastException

☐ Both employees will always be stored

☒ One employee may be treated as a duplicate because compareTo() returns 0

☐ TreeSet does not support Comparable

2. What can happen if a Comparator incorrectly returns 0 for two different objects in a TreeSet?

1 / 1 point

Your Answer: ☒ Correct

☐ Both objects are always stored

☐ TreeSet throws a compilation error

☒ One object may be treated as a duplicate and not stored

☐ The objects are automatically compared using equals()

3. An e-commerce application needs to sort Product objects by price, rating, brand, and discount depending on the screen. Which design is most maintainable?

1 / 1 point

Your Answer: ☒ Correct

☐ Add all conditions to Product.compareTo()

☒ Create multiple Comparator<Product> strategies

☐ Override hashCode() for each requirement

☐ Create a separate Product class for each screen

4. Which statement about Comparator is most accurate?

1 / 1 point

Your Answer: ☒ Correct

☐ A class must implement Comparator to use Collections.sort()

☒ Comparator allows sorting logic to be defined independently of the class

☐ Comparator can only sort primitive values

☐ Comparator is always preferred over Comparable

5. A Java service sorts 1 million Employee objects repeatedly using different fields. Which approach generally provides the cleanest design?

1 / 1 point

Your Answer: ☒ Correct

☐ Put every possible sorting condition inside compareTo()

☒ Create reusable Comparator instances/comparator chains for each business requirement

☐ Convert every employee to a String and sort Strings

☐ Use equals() to determine the sorting order

6. What happens when you try to modify a list created using List.of()?

0.25 / 1 point

```
List<String> names = List.of("Rahul", "Amit", "Priya");  
names.add("Neha");
```

Your Answer: ☒ Incorrect

☐ Element is added successfully

☐ NullPointerException is thrown

☒ Compilation error occurs

☐ UnsupportedOperationException is thrown

7. Which statement correctly differentiates Collections.unmodifiableList() and List.copyOf()?

0.25 / 1 point

Your Answer: ☒ Incorrect

☐ Both create a completely independent mutable copy

☒ Collections.unmodifiableList() provides an unmodifiable view, while List.copyOf() creates an unmodifiable copy

☒ List.copyOf() always allows add() and remove()

☐ Collections.unmodifiableList() and List.copyOf() are both mutable

8. Which statement is correct?

1 / 1 point

Your Answer: ☒ Correct

☐ equals() is used for sorting, while compareTo() is used for hashing.

☐ hashCode() determines object ordering, while compareTo() checks object equality.

☒ equals() checks logical equality, hashCode() supports hash-based collections, compareTo() defines natural ordering, and compareTo() defines external/custom ordering.

☐ compareTo() and compareTo() are identical and can be used interchangeably.

9. class Employee {

 int id;

 String name;

 Employee(int id, String name) {
 this.id = id;

 this.name = name;

 }

 @Override
 public boolean equals(Object obj) {

 if (this == obj) return true;

 if ((obj instanceof Employee)) return false;

 Employee e = (Employee) obj;

 return id == e.id;

 }

 @Override
 public int hashCode() {

 return name.hashCode();

 }

}

0.25 / 1 point

Then:

```
Set<Employee> employees = new HashSet<>();  
employees.add(new Employee(101, "Rahul");  
employees.add(new Employee(101, "Amit");
```

Your Answer: ☒ Incorrect

☐ HashSet behavior is incorrect because objects that are equal must have the same hashCode().

☒ Only one object is stored because equals() returns true.

☐ Both objects are stored because their hashCode() values are different.

☐ Compilation error occurs because equals() and hashCode() cannot use different fields.

10. class Employee {

 int id;

 Employee(int id) {
 this.id = id;

 }

 @Override
 public boolean equals(Object obj) {

 if (this == obj) return true;

 if ((obj instanceof Employee)) return false;

 return id == ((Employee) obj).id;

 }

 @Override
 public int hashCode() {

 return id;

 }

}

0.25 / 1 point

Then:

```
Map<Employee, String> map = new HashMap<>();  
Employee e1 = new Employee(101);  
map.put(e1, "Rahul");  
e1.id = 102;  
System.out.println(map.get(e1));
```

What is the most likely output?

Your Answer: ☒ Incorrect

☒ Rahul

☒ null

☐ 102

☐ ConcurrentModificationException

11. What happens when you structurally modify an ArrayList while iterating using its iterator?

0.25 / 1 point

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));  
for (Integer i : list) {  
  
    if (i == 2)  
        list.remove();  
  
}
```

Your Answer: ☒ Incorrect

☒ Iteration completes successfully

☐ NullPointerException is thrown

☐ The element is safely removed

☒ ConcurrentModificationException is likely thrown

12. What happens in this scenario?

1 / 1 point

```
CopyOnWriteArrayList<Integer> list = new CopyOnWriteArrayList<>  
(List.of(1, 2, 3));  
for (Integer i : list) {  
  
    if (i == 2)  
        list.remove();  
  
}
```

Your Answer: ☒ Correct

☐ ConcurrentModificationException is thrown

☐ The iterator immediately sees the modified list

☒ Iteration can continue because the iterator works on a snapshot

☐ Compilation error occurs

13. A multi-threaded application frequently reads a collection while another thread modifies it. Which option is most appropriate when you specifically need iteration without ConcurrentModificationException and can tolerate snapshot-style iteration?

1 / 1 point

Your Answer: ☒ Correct

☐ ArrayList

☐ HashSet

☐ TreeMap

☒ CopyOnWriteArrayList

14. What is the primary purpose of modCount in Java collections such as ArrayList?

1 / 1 point

Your Answer: ☒ Correct

☐ It stores the total number of elements in the collection

☒ It tracks structural modifications so iterators can detect concurrent modification

☐ It maintains the hash value of each element

☐ It determines the sorting order of elements

15. List<Integer> list = new ArrayList<>(List.of(10, 20, 30));

Iterator<Integer> iterator = list.iterator();

list.add(40);

System.out.println(iterator.next());

0.25 / 1 point

What happens?

Your Answer: ☒ Incorrect

☐ Prints 10 because the iterator always uses the latest list state

☐ Prints 40 because the iterator starts after modification

☒ ConcurrentModificationException is thrown

☒ Compilation error occurs

16. Which statement correctly describes the equals() and hashCode() contract in Java?

1 / 1 point

Your Answer: ☒ Correct

☐ If two objects have the same hashCode(), their equals() method must return true.

☒ If equals() returns true for two objects, their hashCode() values must be the same.

☐ If two objects have different hashCode() values, their equals() method must return true.

☐ Overriding equals() and hashCode() has no impact on HashSet or TreeSet.

17. List<Employee> employees = getEmployees();

Collections.sort(employees);

1 / 1 point

Employee does not implement Comparable<Employee>. What happens?

Your Answer: ☒ Correct

☐ List is sorted using equals()

☐ List is sorted using hashCode()

☐ Compilation error always occurs

☒ ClassCastException occurs at runtime

18. A high-throughput application has many concurrent reads and writes to a shared map. Which is generally the better choice?

1 / 1 point

Your Answer: ☒ Correct

☐ TreeMap

☐ Collections.synchronizedMap(new HashMap<>())

☒ ConcurrentHashMap

☐ HashMap

19. Map<Employee, String> map = new HashMap<>();

Employee e1 = new Employee(101, "Rahul");
map.put(e1, "Developer");

Employee e2 = new Employee(101, "Rahul");
System.out.println(map.get(e2));

Assume Employee correctly overrides both equals() and hashCode() based on id.

What will be printed?

0.25 / 1 point

Your Answer: ☒ Incorrect

☒ null

☒ Developer

☐ Employee@...

☐ ClassCastException

20. Which statement correctly differentiates Iterator and ListIterator?

1 / 1 point

Your Answer: ☒ Correct

☐ Iterator supports bidirectional traversal, while ListIterator supports only forward traversal

☒ Iterator can be used with any Collection, while ListIterator is specifically for List implementations

☐ Both support add() and set() operations

☐ ListIterator cannot remove elements during iteration

21. List<String> names = new ArrayList<>()

List.of("A", "B", "C")

);

ListIterator<String> it = names.listIterator();

while (it.hasNext()) {
 String name = it.next();

 if (name.equals("B")) {
 it.set("X");

 it.add("Y");
 }
}

0.25 / 1 point

What will be the final list?

Your Answer: ☒ Incorrect

☒ [A, B, C]

☒ [A, X, Y, C]

☐ [A, Y, X, C]

☐ ConcurrentModificationException

22. What will be the output of the following code?

1 / 1 point

```
@FunctionalInterface  
interface Calculator {  
  
    int calculate(int a, int b);  
  
    default int calculate(int a) {  
        return calculate(a, 10);  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Calculator c = (a, b) -> a + b;  
  
        System.out.println(c.calculate(5));  
    }  
}
```

0.25 / 1 point

Your Answer: ☒ Correct

☐ Compilation error — functional interface cannot have default methods

☒ 15

☐ Compilation error — lambdas cannot implement an interface with multiple methods

☐ 10

23. Map<Employee, String> map = new HashMap<>();

Employee e1 = new Employee(101, "Rahul");

Employee e2 = new Employee(101, "Rahul");

map.put(e1, "Developer");
map.put(e2, "Tester");

System.out.println(map.size());

1 / 1 point

Assume Employee overrides equals() but does not override hashCode().

What is the most likely output?

Your Answer: ☒ Correct

☐ 1

☒ 2

☐ Compilation error

☐ Depends on insertion order

24. Which statement is correct about the Java Collection Framework?

1 / 1 point

Your Answer: ☒ Correct

☐ Map extends the Collection interface

☒ List, Set, and Queue directly extend Collection

☐ ArrayList directly implements List, Set, and Queue

☐ Map is a subtype of Collection

25. What is the primary reason ArrayList generally provides faster random access than LinkedList?

1 / 1 point

Your Answer: ☒ Correct

☐ ArrayList uses hashing internally

☒ ArrayList stores elements in a dynamically resizable array

☐ LinkedList does not allow indexed access

☐ ArrayList is synchronized by default

26. Which statement best explains why a HashSet does not allow duplicate elements?

1 / 1 point

Your Answer: ☒ Correct

☐ It uses compareTo() to identify duplicates

☐ It uses only == to compare elements

☒ It uses hashCode() and equals() to determine logical duplicates

☐ It internally uses a TreeSet

27. Which statement is most accurate?

1 / 1 point

Your Answer: ☒ Correct

☒ Comparable is used to define natural ordering, while Comparator can define external/custom ordering

☐ Comparator must always be implemented by the class being sorted

☐ Comparable supports multiple different sorting strategies simultaneously

☐ Comparator can only be used with TreeSet

28. Which scenario can cause a memory leak in a Java application even though Java has Garbage Collection?

1 / 1 point

Your Answer: ☒ Correct

☐ Creating many local variables inside a method

☒ Keeping unused objects referenced in a long-lived static collection

☐ Calling System.gc() frequently

☐ Using ArrayList instead of LinkedList

29. Which statement about Garbage Collection in Java is correct?

0.25 / 1 point

Your Answer: ☒ Incorrect

☐ Calling System.gc() guarantees immediate garbage collection

☒ An object becomes eligible for GC immediately after null is assigned to any variable

☒ An object is eligible for GC when it is no longer reachable from GC roots

☐ Garbage Collection primarily removes objects from the stack

30. public void process() {

 List<String> names = new ArrayList<>();

 names.add("Java");

}

1 / 1 point

Which statement is most accurate?

Your Answer: ☒ Correct

☐ Both names and the ArrayList object are stored entirely on the stack

☒ names is a local reference associated with the stack frame, while the ArrayList object is allocated on the heap

☐ names is stored on the heap and the ArrayList is stored on the stack

☐ Both are always stored in the heap

Logout